



Desarrollo de Aplicaciones Informáticas

Examen - NextJS con Sockets

Docente: Ing. Pablo Morandi

Ayudante: Matías Marchesi

Curso: 5to año - Informática

Duración: 2 horas cátedra (80 min)

Fecha: Febrero 2026

EXAMEN RECUPERATORIO - PERÍODO FEBRERO

Leer con atención antes de empezar el examen:

Material permitido: Apuntes de clase, documentación oficial. El uso de la IA implica que **el examen sea anulado automáticamente** sin excepción y con nota 1(unos).

Formato de entrega: Subir carpeta completa del proyecto en un archivo .RAR y con el formato "5x_DAI_APELLIDO" a Google Classroom.

POR FAVOR, respetar el formato de entrega.

IMPORTANTE: la carpeta **/node_modules** y la carpeta **./next** **NO DEBEN SER ENTREGADAS**. De ser así, **se restará 1 (uno) punto** de la nota final del examen sin excepción.

Sistema de Votación en Tiempo Real

Desarrollar una aplicación de votación donde múltiples usuarios votan por opciones y ven resultados actualizados en tiempo real usando **Socket.IO**, **Router** para navegación, **Conditional Rendering** para mostrar diferentes estados y **useState** y **useEffect** para manejo de datos.

Ejercicio 1: Página de Inicio con Router

Crear una página de inicio en `/inicio` que cumpla con los siguientes requisitos:

Requisitos

1. Formulario de ingreso:

- Input para el nombre del votante (`votante`) (mínimo 3 caracteres)
- Input para el ID de alumno (`alumnoId`). Este ID es el nro que a usted le corresponde de la lista del curso ordenada alfabéticamente.
- Botón "Acceder a la Votación"

2. Validación y estados:

- Mostrar mensaje de error si el nombre tiene menos de 3 caracteres
- El mensaje de error debe mostrarse usando **Conditional Rendering**

3. Navegación con Router:

- Al hacer clic en "Acceder a la Votación", redirigir a `/votacion`
- Pasar `votante` y `alumnoId` como query params
 - URL: `http://localhost:3000/votacion?votante=Juan&alumnoId=4`

Ejercicio 2: Creación de componentes

2a) Crear un componente `OpcionVoto.js` que:

1. Reciba por props: `id` (string), `nombre` (string), `votos` (number), `porcentaje` (number)

2. Conditional Rendering:

- Mostrar el porcentaje
- Mostrar el nombre
- Si `votos === 0`: mostrar "Sin votos"
- Si `porcentaje > 50`: mostrar "🏆 GANADORA"

2b) Crear un componente `ResultadosVotacion.js` que:

1. Reciba por props: `opciones` (array)

2. El componente debe tener:

- Título "Resultados en tiempo real"
 - Usar `.map()` o iterar sobre las opciones para mostrar cada una
 - Usar el componente `OpcionVoto` para cada opción
-

Ejercicio 3: Página de Votación con Socket.IO

Crear una página de votación en `/votacion` que implemente comunicación en tiempo real.

Backend Provisto

Servidor: `http://localhost:4000`

Eventos Socket.IO

Eventos que ENVIÁS:

Evento	Datos
<code>join_votacion</code>	<code>{ alumnoId }</code>
<code>emitir_voto</code>	<code>{ votante, opcion }</code>

Nota: `opcion` puede ser "A", "B", "C" o "D"

Eventos que RECIBÍS:

Evento	Datos
<code>joined_OK_votacion</code>	<code>{ room, votacion }</code>
<code>votacion_actualizada</code>	<code>{ votacion }</code>
<code>votacion_finalizada</code>	<code>{ votacion }</code>
<code>error_votacion</code>	<code>{ mensaje }</code>

```
// Datos que recibís (ejemplo):
{
  room: number,
  votacion: {
    pregunta: string,
    opciones: [
      { id: "A", nombre: "JavaScript", votos: 0, porcentaje: 0 },
      { id: "B", nombre: "Python", votos: 0, porcentaje: 0 },
      { id: "C", nombre: "Java", votos: 0, porcentaje: 0 },
      { id: "D", nombre: "C#", votos: 0, porcentaje: 0 }
    ],
    totalVotos: number,
    opcionGanadora: "A" | "B" | "C" | "D",
    timestamp: string
  }
}
```

Requisitos Frontend

1. Conexión con Socket.IO

- Conectar a `http://localhost:4000`
- Obtener `votante` y `alumnoId` desde query params

2. Unirse a la votación

- Botón "Conectarse a la Votación"
- Emitir `join_votacion` con `{ alumnoId }`
- Escuchar `joined_OK_votacion` y guardar votación inicial en un estado.

3. Emitir votos

- Crear 4 botones (Opción A, B, C, D)
- Al hacer clic en un botón, emitir `emitir_voto` con `{ votante, opcion }`
- La opcion debe ser "A", "B", "C" o "D"

4. Escuchar actualizaciones

- Evento `votacion_actualizada`: Actualizar resultados con componente `ResultadosVotacion`
- Evento `votacion_finalizada`: Mostrar mensaje "¡Votación finalizada! Ganadora: Opción {opcionGanadora}"
- Evento `error_votacion`: Mostrar alert con el error

5. Conditional Rendering

- Mostrar "Sala: {alumnoId}" cuando conectado
 - Mostrar "Votante: {votante}"
 - Mostrar "Pregunta: {pregunta}"
 - Mostrar "Votos: {totalVotos}"
 - Deshabilitar botones de voto si no está conectado o si votación finalizó
-

Ejercicio 4: Implementar Evento del Backend

IMPORTANTE: En este ejercicio debes escribir TODO el código del evento desde cero. **IMPORTANTE:** Ver objeto de respuesta del socket del ejercicio anterior.

Crear un archivo `evento-sockets.js` con el siguiente evento completo:

Implementar el evento `reiniciar_opcion` completo

```
socket.on("reiniciar_opcion", ({ opcion }) => {
  const ROOM = socket.data.room;

  if (!ROOM || !votacionSalas[ROOM]) {
    socket.emit("error_votacion", { mensaje: "No estás en una sala" });
    return;
  }

  const votacion = votacionSalas[ROOM];

  // TODO 1: Validar que la opción sea válida ("A", "B", "C" o "D")
  // Si no es válida, emitir 'error_votacion' con { mensaje: "Opción inválida" } y
  return
  // COMPLETAR ACÁ:

  // TODO 2: Confirmar SOLO al usuario que reinició
  // Emitir 'opcion_reiniciada' con { mensaje: "Opción reiniciada" }
  // COMPLETAR ACÁ:

  // TODO 3: Notificar a TODA LA SALA la votación actualizada
  // Emitir 'votacion_actualizada' con { votacion: votacionSalas[ROOM] }
  // COMPLETAR ACÁ:

});
```